

# Volumes (Databricks Unity Catalog)

## What are Volumes?

Volumes are Unity Catalog objects in Databricks that provide governance, access control, and management over non-tabular datasets.

While tables are used to govern tabular data, volumes are used to govern file-based, non-tabular data of any format.

Databricks strongly recommends using Volumes to govern all non-tabular data instead of directly accessing cloud storage paths.

## Types of Data Governed by Volumes

Volumes can store and govern:

- Structured data (CSV, Parquet files not registered as tables)
- Semi-structured data (JSON, XML, Avro)
- Unstructured data (images, PDFs, audio, video, text files, logs)

Any file-based data that is not represented as a table should be governed using a volume.

## Why Use Volumes?

Volumes provide the following benefits:

- Centralized governance through Unity Catalog
- Fine-grained access control
- Auditing and lineage support
- Secure access without exposing cloud storage paths
- Seamless integration with Databricks notebooks, jobs, and workflows

## Types of Volumes

Databricks supports two types of volumes:

### 1. Managed Volumes

Managed volumes are fully managed by Databricks. Databricks controls the storage location and lifecycle of the data.

### Key Characteristics:

- No cloud storage path needs to be specified
- Storage and folder structure are managed by Databricks
- Strong governance by default
- Ideal for internally generated data

### Typical Use Cases:

- Application-generated files
- Machine learning artifacts
- Intermediate datasets
- Logs and checkpoint files

## 2. External Volumes

External volumes reference an existing cloud storage location while enabling governance through Unity Catalog.

### Key Characteristics:

- Data remains in customer-managed cloud storage
- No data movement required
- Unity Catalog controls access and permissions
- Suitable for legacy or shared datasets

### Typical Use Cases:

- Pre-existing cloud storage data
- Data shared across multiple platforms
- External systems writing directly to cloud storage

### External Volume Prerequisite: External Location

An external location defines a trusted cloud storage path that Unity Catalog can access.

#### Example (Azure Data Lake Storage Gen2)

- **External Location Name:** externaldata
- **Container / Folder:** ext\_volume
- **URI:**

abfss://externaldata@ttmystorageaccount001.dfs.core.windows.net/ext\_volume

Once the external location is configured, an external volume can be created on top of a subfolder within that location.

## Key Characteristics of External Volumes

- Data physically resides in customer-managed cloud storage
- Unity Catalog governs access permissions
- No data movement is required
- External tools and systems can still access data using direct URIs
- Enables controlled access at subfolder level

### Managed vs External Volumes

| Feature              | Managed Volume                  | External Volume                              |
|----------------------|---------------------------------|----------------------------------------------|
| Storage Location     | Databricks-managed              | Customer-managed cloud storage               |
| Data Lifecycle       | Managed by Databricks           | Managed externally                           |
| Access Control       | Fully enforced by Unity Catalog | Governed by UC, but external access possible |
| External Tool Access | Not allowed                     | Allowed via direct URIs                      |
| Typical Use Case     | Databricks-only workloads       | Mixed Databricks + external systems          |

## Access Control Differences

### Managed Volumes

- All access goes through Unity Catalog
- No direct access using cloud storage URIs
- Strongest governance and isolation

## External Volumes

- Unity Catalog controls access within Databricks
- External systems can still access data using cloud URIs
- Useful when data must be shared across platforms

## Why External Volumes When External Locations Already Exist?

This is a key conceptual distinction.

### External Locations

- Grant access to a root cloud storage path
- Permissions apply to the entire location
- Coarse-grained access control

### External Volumes

- Are created within external locations
- Allow governance at subfolder level
- Enable granular access control

#### In short:

- External locations control where Databricks can access data
- External volumes control who can access which part of that data

This enables fine-grained governance without exposing the entire storage path.

## DBFS Mount Points vs Unity Catalog Volumes

Historically, Databricks used DBFS mount points to access external cloud storage.

Databricks is now transitioning from DBFS mount points to Unity Catalog volumes because volumes provide:

- Centralized governance
- Fine-grained access control
- Better auditing and lineage
- Native integration with the Unity Catalog security model
- Stronger alignment with the broader Databricks ecosystem

Volumes are the recommended replacement for DBFS mount points.

## **Databricks Platform Architecture Overview**

Databricks provides a unified data and AI platform built on the following core pillars:

- Lakeflow – Data ingestion, transformation, and orchestration
- Data Processing Engine – Apache Spark
- Unified Governance – Unity Catalog
- Optimized Storage – Delta Lake

These components work together to enable scalable, governed, and reliable data pipelines.

### **Apache Spark (Data Processing Engine)**

Databricks uses Apache Spark as its underlying distributed data processing engine.

Spark is responsible for:

- Large-scale data processing
- Batch and streaming workloads
- SQL, ETL, and machine learning computations

All higher-level Databricks features (Lakeflow, Delta Lake, DLT) are executed on top of Spark.

### **Lakeflow Overview**

Lakeflow is Databricks' unified framework for building and managing data pipelines.

It integrates:

- Data ingestion
- Data transformation
- Workflow orchestration

All Lakeflow components run on:

- Apache Spark (processing engine)
- Delta Lake (storage layer)
- Unity Catalog (governance layer)

### **Lakeflow Components**

Lakeflow consists of three core components:

1. Lakeflow Connect – Data ingestion
2. Lakeflow Declarative Pipelines – Data transformation using Delta Live Tables (DLT)

### 3. Lakeflow Jobs – Workflow orchestration and scheduling

## Lakeflow Connect (Data Ingestion)

### Background

Traditionally, organizations relied on:

- Multiple third-party ingestion tools
- Custom-built ingestion frameworks
- Separate tools for different data sources

This led to:

- Tool sprawl
- Increased operational overhead
- Inconsistent governance

### Lakeflow Connect

Lakeflow Connect provides a native Databricks solution for ingesting data from multiple sources into Delta Lake.

Supported source types include:

- Cloud storage (ADLS Gen2, Amazon S3, Databricks Volumes)
- Databases
- SaaS applications

All ingested data lands in Delta Lake Bronze tables.

Lakeflow pipelines typically follow the Medallion Architecture,

- Bronze – Raw ingested data
- Silver – Cleaned and enriched data
- Gold – Aggregated, business-ready data
- Consumers – BI, analytics, ML, applications

### Lakeflow Declarative Pipelines (Delta Live Tables)

Delta Live Tables (DLT) are used for transforming data between:

- Bronze → Silver
- Silver → Gold

Key characteristics:

- Declarative transformation definitions
- Built-in data quality enforcement
- Automatic dependency resolution
- Native Delta Lake integration (assumed knowledge)

**Lakeflow Jobs** are used to:

- Schedule ingestion and transformation pipelines
- Orchestrate end-to-end workflows
- Monitor execution and failures

Jobs integrate ingestion, DLT pipelines, and other Databricks tasks into a single workflow.

## **Lakeflow Connect – Connector Types**

Lakeflow Connect is Databricks' native data ingestion framework that enables building scalable, governed, and efficient ingestion pipelines directly within the Databricks platform.

It is used to ingest data from multiple source systems into Delta Lake, without relying on external third-party ingestion tools.

### **Types of Connectors**

1. **Upload Connectors**
  - Upload files from local systems into Databricks volumes
2. **Standard Connectors**
  - Ingest data from cloud storage into Delta Lake
3. **Managed Connectors**
  - Ingest data from SaaS applications and databases

### **Standard Connectors – Ingestion Modes**

Standard connectors support:

- **Batch ingestion**  
Re-ingests full data on every run
- **Incremental batch ingestion**  
Ingests only new data; skips already ingested records

- **Incremental streaming ingestion**  
Continuously ingests data in near real time

## Data Ingestion from Cloud Storage

This section explains the approaches used to ingest data from cloud storage into Delta Lake using Lakeflow Connect standard connectors.

Standard connectors are responsible for converting raw files stored in cloud object storage into Delta tables, which then participate in downstream ETL and analytics workflows.

Data ingestion can be performed from the following cloud storage systems:

- Azure Data Lake Storage Gen2
- Amazon S3
- Databricks Volumes

The ingested data is stored in Delta Lake, typically in the Bronze layer of the Medallion Architecture.

Databricks provides multiple ingestion mechanisms for cloud storage data, each suited for different workload patterns.

### 1. CTAS (Create Table As Select)

CTAS is a SQL-based ingestion mechanism used to create a Delta table directly from source data.

```
CREATE TABLE target_table  
AS SELECT * FROM source;
```

- The table is created if it does not already exist
- Data is loaded immediately during table creation

CTAS is commonly used when:

- Performing initial data ingestion
- Loading raw files from cloud storage
- Creating Delta tables quickly for exploration or downstream processing
- Data volume is manageable and full reloads are acceptable

CTAS is typically used in batch ingestion scenarios.

### CTAS Ingestion Behavior



## Full Data Reload

- All data is re-ingested every time the CTAS statement is executed
- There is no built-in tracking of previously ingested data
- Existing tables are either overwritten or recreated (depending on usage)

Implication:

CTAS is not suitable for recurring ingestion of large datasets where incremental loading is required.

## CTAS and Cloud Storage

CTAS is commonly used to ingest data from:

- Azure Data Lake Storage Gen2
- Amazon S3
- Databricks Volumes

Raw files such as CSV, JSON, or Parquet are converted into Delta tables, usually landing in the Bronze layer of the Medallion Architecture.

## Handling Bad or Malformed Records in CTAS

When ingesting raw files using CTAS, Spark may encounter malformed records. The mode option controls how Spark behaves in such cases.

### Permissive Mode

- Default behavior
- Job continues even if malformed records are encountered
- Bad records are stored in the `_rescued_data` column

Recommended for:

Initial ingestion where data quality issues are expected.

### DropMalformed Mode

- Malformed records are silently dropped
- Job continues without failure
- No visibility into dropped records

Recommended for:

Non-critical data where occasional loss is acceptable.

### FailFast Mode

- Job fails immediately upon encountering the first malformed record
- Ensures strict data correctness

Recommended for:

Scenarios where data quality is non-negotiable.

## **COPY INTO**

COPY INTO is a Databricks-native command used to load data from cloud file locations into Delta Lake tables.

It is designed for reliable, incremental batch ingestion and addresses the limitations of full reload approaches such as CTAS.

### **Key Characteristics of COPY INTO**

- Loads data from cloud storage into Delta Lake
- Idempotent and retrievable operation
- Guarantees exactly-once processing
- Automatically skips files that have already been ingested
- Optimized for ingestion of up to thousands of files

COPY INTO is particularly suitable for recurring ingestion workloads.

### **Incremental Batch Ingestion Behavior**

COPY INTO follows an incremental batch ingestion model:

- Only new files in the source location are ingested
- Files that have already been loaded are automatically skipped
- No manual tracking of processed files is required

This makes COPY INTO more efficient and safer than full reload ingestion methods.

## **Table Creation Options**

### **Creating an Empty Table Without Schema**

COPY INTO allows ingestion into a table that is created without explicitly defining a schema.

In this case:

- Schema is inferred from the source data
- Useful when schema is not known in advance

## Creating a Table With Schema

Alternatively, a Delta table can be created with a predefined schema before running COPY INTO.

This approach:

- Enforces schema consistency
- Prevents unexpected schema drift
- Is recommended for production pipelines

## COPY INTO Configuration Options

COPY INTO provides two important configuration categories that control ingestion behavior.

### **format\_options()**

format\_options() define how source files are parsed and interpreted.

Examples include:

- Delimiter settings
- Header presence
- Date and timestamp formats
- Character encoding

These options control file-level parsing behavior.

### **copy\_options()**

copy\_options() control the behavior of the COPY INTO operation itself.

Key capabilities include:

- Schema evolution support (e.g., mergeSchema)
- Handling of schema changes during ingestion
- Operational behavior of the copy process

These options influence how data is written into Delta Lake.

COPY INTO provides strong ingestion guarantees:

- **Exactly-once processing**  
Each file is ingested only once, even if the command is retried
- **Retry-safe execution**  
Failed or interrupted COPY INTO operations can be safely rerun

These guarantees make COPY INTO suitable for production workloads.

## When to Use COPY INTO

COPY INTO is best suited when:

- Data arrives periodically in cloud storage
- Incremental ingestion is required
- File counts range from tens to thousands
- Streaming ingestion is not necessary
- Reliability and idempotency are important

COPY INTO works well for incremental batch ingestion when dealing with up to thousands of files. However, it becomes inefficient or impractical in the following scenarios:

- New files arrive every few minutes or seconds
- Millions of files arrive every hour
- Large-scale backfill or migration involving billions of files
- Near real-time ingestion is required

To handle these high-scale and high-frequency ingestion scenarios, Auto Loader is used.

## Auto Loader?

Auto Loader is an advanced ingestion mechanism in Databricks designed to incrementally and efficiently process new data files as they arrive in cloud storage.

It provides:

- High scalability
- Exactly-once processing
- Support for both batch-style and streaming-style ingestion

Auto Loader is more advanced and scalable than COPY INTO.

## Key Capabilities of Auto Loader

- Incrementally processes newly arriving files
- Supports incremental batch and streaming ingestion
- Can backfill or migrate billions of files
- Scales to ingest millions of files per hour
- Guarantees exactly-once semantics
- Handles failures and restarts gracefully

## Supported Cloud Storage Sources

Auto Loader can ingest data from:

- Amazon S3
- Azure Data Lake Storage Gen2
- Google Cloud Storage (GCS)
- Databricks Volumes

Example Volume Path:

`/Volumes/deltalake_catalog/default/raw/ordershistory/`

This path represents:

- A Unity Catalog-governed volume
- A folder (ordershistory) containing raw data files

## Auto Loader Ingestion Modes

Auto Loader supports both batch-style and streaming-style ingestion using a unified API.

Unified API: `writeStream`

The `writeStream` API is used for both batch and streaming ingestion.

## Streaming Mode

Streaming mode continuously processes new files as they arrive.

`.trigger(processingTime = "10 seconds")`

## Characteristics:

- Near real-time ingestion
- Suitable for frequently arriving data
- Processes data in micro-batches

## Batch-Style Mode (Available Now)

Batch-style ingestion processes all available files and then stops.

`.trigger(availableNow = True)`

### Characteristics:

- Used for backfills or large migrations
- Processes all pending files once
- No continuous execution

## How Auto Loader Tracks Ingestion Progress

Auto Loader tracks ingestion progress using a checkpoint mechanism.

### Metadata Tracking

- As files are discovered, their metadata is stored in a scalable key-value store
- This store is implemented using RocksDB
- Metadata is persisted in the checkpoint location

## Exactly-Once Semantics

Because file metadata is stored in the checkpoint:

- Each file is processed only once
- In case of job failure, Auto Loader resumes from the last processed state
- No duplicate data is ingested

This mechanism is what enables exactly-once processing guarantees.

## Failure Recovery

If an Auto Loader job fails:

- Previously processed files are remembered
- Processing resumes from the last successful point
- No manual intervention is required

This makes Auto Loader suitable for production-grade ingestion pipelines.

## Auto Loader – Schema Management

When ingesting data using Auto Loader, schema management is a critical aspect. Auto Loader provides flexible mechanisms to:

- Infer schema automatically
- Manually define schema
- Control schema evolution behavior
- Capture unexpected or malformed data
- Add ingestion-related metadata columns

This section explains how schema inference works internally, how to control it, and how Auto Loader reacts to schema changes.

### 1. How Schema Inference Works Internally

When schema inference is enabled, Auto Loader does not scan all files in the source directory. Instead, it infers schema based on a sample of data.

#### Sampling Limits for Schema Inference

Schema inference stops when either condition is met first:

- 1000 files
- 50 GB of data

Examples:

- 1000 files x 1 MB each = 1 GB → inference stops at 1000 files
- 10 files x 5 GB each = 50 GB → inference stops at 50 GB

This design ensures schema inference is efficient and scalable, even for large datasets.

### Configuration Options for Schema Inference

You can control how much data Auto Loader samples using the following options:

- `cloudFiles.schemaInference.sampleSize.numBytes`  
Controls the maximum number of bytes scanned for schema inference.
- `cloudFiles.schemaInference.sampleSize.numFiles`  
Controls the maximum number of files scanned for schema inference.

These options allow fine-grained tuning when dealing with very large or very small files.

## 2. Manually Defining the Schema

In some scenarios, schema inference may not be desirable, such as:

- Strict schema enforcement is required
- Schema is already known in advance
- Avoiding unexpected schema changes in production pipelines

In such cases, you can explicitly define the schema while configuring Auto Loader.

### Important Behavior Change

- When a schema is not provided, Auto Loader defaults to “add new columns” mode.
- When a schema is provided, Auto Loader defaults to none mode.

This distinction is important for understanding schema evolution behavior.

## 3. Schema Hints

Schema hints allow you to partially control schema inference without fully defining the schema.

### Use Cases for Schema Hints

- Enforcing correct data types for specific columns
- Preventing incorrect inference (for example, numeric columns inferred as strings)
- Combining flexibility of inference with partial control

Schema hints act as overrides or guides during schema inference rather than replacing it entirely.

## 4. Adding New Metadata Columns

Auto Loader allows adding system-generated metadata columns during ingestion.

Typical metadata columns include:

- File path
- File name
- File modification time
- Ingestion timestamp

These columns are useful for:

- Auditing
- Debugging
- Incremental processing logic



- Data lineage and observability

## Schema Evolution Modes in Auto Loader

Schema evolution determines how Auto Loader reacts when new columns appear in incoming data.

Available modes:

- none
- failOnNewColumns
- rescue
- addNewColumns (default)

### 1. none

**Behavior:**

- Schema is not evolved
- New columns are ignored
- Data is not rescued unless rescuedDataColumn is explicitly set
- Stream does not fail due to schema changes

**Use Case:**

- Strict schema enforcement
- Downstream systems cannot tolerate schema changes

### 2. fail on new columns

**Behavior:**

- Stream fails immediately when new columns are detected
- Stream does not restart unless:
  - Schema is updated manually, or
  - The offending data file is removed

**Use Case:**

- Strong data quality guarantees
- Schema changes must be reviewed before ingestion

### 3. rescue

**Behavior:**

- Schema is never evolved
- Stream does not fail due to schema changes
- All new or unexpected columns are captured in the rescued data column

**Use Case:**

- Ingest first, analyze schema changes later
- Handling semi-structured or evolving data sources safely

#### **4. add new columns (default)**

**Behavior:**

- Stream detects new columns
- New columns are added to the schema automatically
- Existing columns do not evolve their data types
- Stream may briefly fail and then restart with updated schema

**Default Rules:**

- Default mode when no schema is provided
- Not the default when a schema is explicitly defined

**Use Case:**

- Rapidly evolving data sources
- Bronze-layer ingestion in Medallion architecture

**Key Defaults to Remember**

- No schema provided → addNewColumns is the default
- Schema provided → none is the default

**This design encourages:**

- Flexibility during early ingestion
- Strict control when schemas are explicitly managed